

Hardware

Nano Matter

Tutorials

Nano Matter User Manual

Getting Started with Nano Matter Display

Matter Smart Fan with the Arduino Nano Matter

Matter Smart Relay with the Arduino Nano Matter

Matter RGB Light with the Arduino Nano Matter

Matter Temperature Sensor with the Arduino Nano Matter

Open Thread Border Router with Nano Matter & ESP32

ML Magic Wand with the Arduino Nano Matter

Home / Hardware / Nano Matter / **Open Thread Border Router with Nano Matter & ESP32**

Open Thread Border Router with Nano Matter & ESP32

Learn how to create your own Thread Border Router using OpenThread and Arduino ecosystem products.

Author: Martino Facchin, Leonardo Cavagnis and Christopher Méndez
Last revision: 02/17/2025

ON THIS PAGE

Introduction

OpenThread Border
Router

Goals

Hardware and Software Requirements

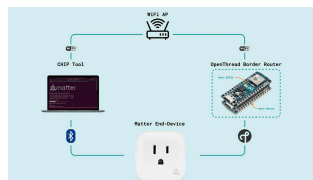
Setting up the OTBR +

The Use Case: Smart Outlet +

Conclusion +

Introduction

Thread is a low-power, wireless mesh networking protocol for smart homes and Internet of Things (IoT) devices. A Thread Border Router serves as a bridge between the Thread network and the wider Internet or local networks, allowing devices within the Thread network to communicate with external systems.



Project overview

Matter devices can use Thread as their primary communication method, especially for low-power

Help

Thread protocol and leverage Matter's application layer for interoperability.

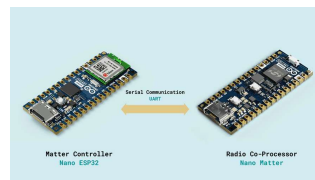
OpenThread Border Router

An OpenThread Border Router (OTBR) consists of a **Matter Controller** and a **Radio Co-Processor** (RCP):

The *Matter Controller* is essential for managing devices using the Matter protocol, which ensures interoperability between nodes. It handles: commissioning, communication and network management.

The *Radio Co-Processor* (RCP) is dedicated to handling Thread network communications, improving efficiency by offloading radio communication tasks.

The **Nano Matter** is the **RCP**, connected to the **Nano ESP32** (the Matter Controller) via serial port.



OpenThread border router architecture

Goals

This tutorial's main objective is to guide you through the build and configuration of an OpenThread Border Router. This router will allow you to deploy a Matter network over Thread and integrate Matter devices into your Smart Home system.

Leverage the Nano Matter as a Radio Co-Processor (RCP).

Use the Nano ESP32 as a Matter Controller.

Integrate a smart outlet based on the Nano Matter to your network.

Hardware and Software Requirements

Hardware Requirements

[Nano Matter](#) (x2)

[Nano ESP32](#) (x1)

Linux Computer (Laptop/PC)
(x1)

[USB-C® cable](#) (x1)

Software Requirements

[Arduino IDE 2.0+](#)

[Silicon Labs Core](#)

[Simplicity Studio](#)

[Visual Studio Code](#)

Setting up the OTBR

The RCP: Nano Matter

This section outlines the steps to build the RCP firmware for the Nano Matter.



Nano Matter Configuration

Ready to Flash Binary

We recommend you to use this pre-compiled binary to flash your Nano Matter board.

Download the `.hex` binary from [here](#) and locate it on a known directory.

Open any text editor and paste the following command:

```
1 /Users/<username>/AppData/
```

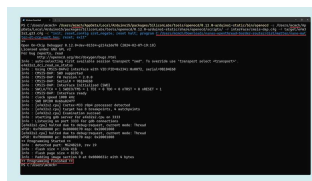
Update the `username` field with yours.

Update the `project-directory` field with the binary directory you downloaded in the previous step.

Open the Command Prompt and paste the formatted command.

Connect the Nano Matter to your PC using a USB-C® cable.

Run the command and verify the download process was successful.



Binary flash process



Make sure to have the [Silicon Labs core](#) installed on your Arduino IDE so you can use the **openocd** tool for flashing.

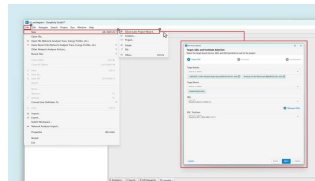
From here, jump directly to the [Matter Controller section](#).

Build the Binary From Scratch (Optional)

If you want to build the Nano Matter program by yourself, follow the steps below:

Download Simplicity Studio.
Silicon Labs provides this IDE, which is designed to simplify the development process for Silicon Labs hardware platforms. Download latest version [here](#).

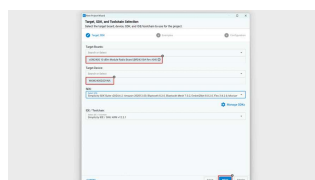
Open Simplicity Studio and create a new project by clicking **File > New > Silicon Labs Project Wizard**.



New project creation

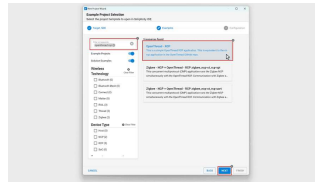
Set the following project configurations and click on *Next*.

Field	Setting
Target Boards	xGM240S 10 dBm Module Radio Board (BRD4318A Rev A04)
Target Device	MGM240SD22VNA



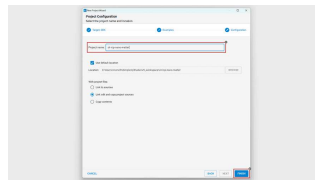
Project configuration

example, select it and click on *Next*.



Example project selection

In the *Configuration* tab give the project a name and click on *Finish*.

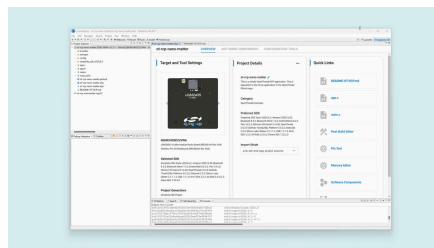


Final project configuration

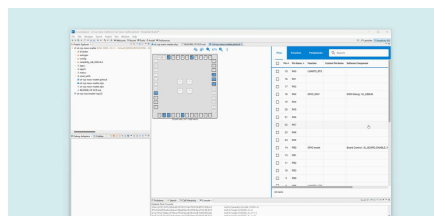
Change the Serial Port Pinout

The default pinout is **PA8 > TX** and **PA9 > RX**, but we need to change it to **PA4 > TX** and **PA5 > RX**, so they correspond to the **D0** and **D1** of the standard Nano layout.

Delete the default `USART0` setting:



Add the new setting:



Save your project with the new configurations.

Disable USART Flow Control

Open the

```
<project-  
folder>/config/sl_uartdrv_  
usart_vcom_config.h
```

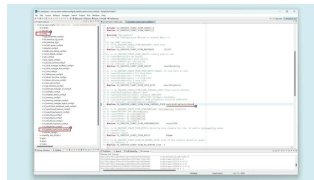
file.

Modify the

```
SL_UARTDRV_USART_VCOM_FLOW_  
_CONTROL_TYPE
```

variable as follows:

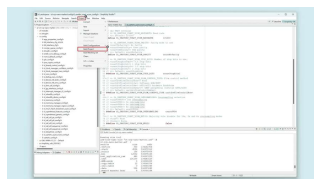
```
1 #define SL_UARTDRV_USART_V
```



Flow Control Disabled

Build and Flash the Project

In the upper menu navigate to **Project > Build Project** to compile and build the RCP firmware.



Project Build

In your project directory
navigate to the

```
/GNU ARM v12.2.1 -  
Default/
```

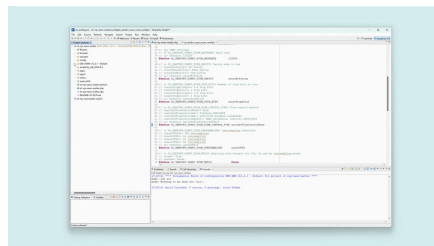
folder, right click on it and open
a **Command Line**

Use the following command to flash the firmware to the Nano Matter, make sure to modify the

<username> and <project name>

with yours:

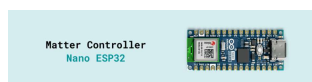
```
1 /Users/<your-username>/App
```



The **Openocd** tool directory may vary according to your OS, modify the command above respectively.

The Matter Controller: Nano ESP32

This section outlines the steps to build the Matter Controller firmware for the Arduino Nano ESP32.



Nano ESP32 Configuration

To set up the environment for the ESP32 firmware development use the following commands on a **Linux computer**.

Install dependencies (Ubuntu and Debian):



Search for the correct commands [here](#) if you use a different Linux distribution.

Install the **ESP-IDF**:

```
1 git clone -b v5.1.2 --recurse-submodules esp-idf
2 cd esp-idf
3 ./install.sh
4 . ./export.sh
```

Navigate to the `ot_rcp` example included in the `esp-idf` directory, and build it using the `esp32h2` target:

```
1 cd examples/openthread/ot_rcp
2 idf.py set-target esp32h2
3 idf.py build
```

Clone the ESP Thread Border Router example repository:

```
1 cd ..
2 git clone -b v1.0 --recursive esp-thread-br
3 cd esp-thread-br/examples/
```

For the `sdkconfig` file to be generated, and we can later modify it with our Nano ESP32 custom settings, set the device target from the command line:

```
1 idf.py set-target esp32s3
```

Using a **code editor**, open the

`sdkconfig` file located in

```
esp-thread-  
br/examples/basic_thread_border  
_router/sdkconfig
```

and do the following modifications:

Update the Wi-Fi credentials
with your network `SSID` and
`Password`:

```
1 CONFIG_EXAMPLE_WIFI_SSID="  
2 CONFIG_EXAMPLE_WIFI_PASSWO
```

Modify the Serial pinout so it
matches with the Arduino Nano
layout:

```
1 CONFIG_PIN_TO_RCP_TX=43  
2 CONFIG_PIN_TO_RCP_RX=44
```

Disable the RCP firmware auto-
update verifying this parameter
is not set:

```
1 # CONFIG_OPENTHREAD_BR_AUT
```

Enable the OpenThread
webserver:

```
1 CONFIG_OPENTHREAD_BR_START
```

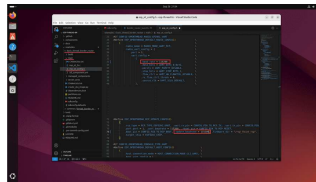
Enable OpenThread
commissioner and joiner:

```
1 CONFIG_OPENTHREAD_COMMISSIONING=y  
2 CONFIG_OPENTHREAD_JOINER=y
```

Navigate to

```
esp-thread-  
br/examples/basic_thread_b  
order_router/main/esp_ot_c  
onfig.h
```

Modify the Serial port baud rate to `115200`, the result should look like this:

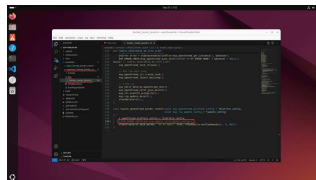


Baud rate configuration
update

Navigate to

```
esp-thread-  
br/examples/common/thread_  
border_router/src/border_r  
outer_launch.c
```

Disable the ESP RCP update process by commenting the following line of the `border_router_launch.c` file, the result should look like this:



RCP auto update disabling

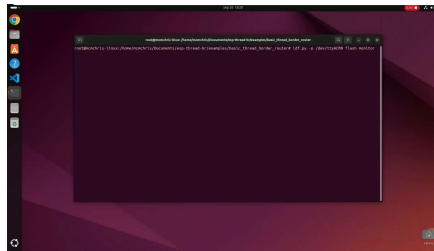
Go back to the

```
basic_thread_border_router  
example folder and build the  
firmware:
```

```
1 cd esp-thread-br/examples/  
2 idf.py build
```

When the build is completed,
flash the Nano ESP32:

```
1 idf.py -p /dev/ttyACM0 fla
```



Do not run the

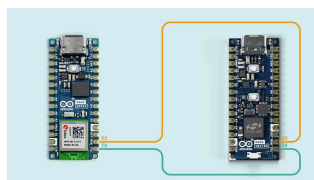
```
idf.py set-target  
esp32s3
```



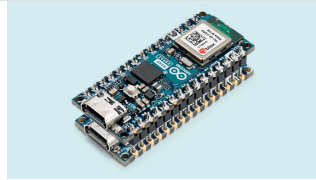
command again to avoid
overriding the customized
configurations.

OTBR Assembly

After configuring and flashing the
Nano Matter and Nano ESP32
separately, it is time to connect them.
As they communicate through a
serial connection, you can stack them
using breakout headers or jumper
wires as follows:



Nano ESP32 + Nano Matter
wiring



Nano ESP32 + Nano Matter
stacking



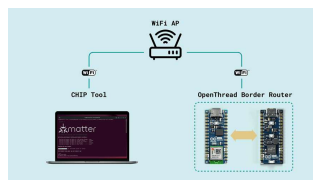
It doesn't matter which board is on top.

CHIP Tool

CHIP Tool is a command-line tool for *commissioning, controlling, and managing Matter* devices within a Matter network.

Due to its high demand for storage space and computational power it must be executed on a separate, more powerful device, such as a Unix laptop (macOS or Linux), that is connected to the same Wi-Fi network.

This third device will handle the complex tasks, while the Nano ESP32 acts as the Matter Controller in the network.



Network layout

Configure the CHIP Tool

On a Linux system, clone the CHIP Tool repository:

```
1 git clone --recurse-submod
```

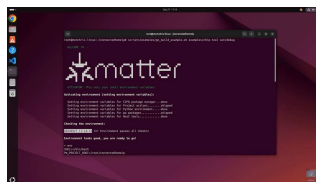
Install prerequisites:

Before building, you must install a few OS specific dependencies.

```
1 sudo apt-get install git g
2     libglib2.0-dev libava
3     python3-pip unzip lib
4     default-jre
```

Open a command prompt in the `connectedhomeip` directory and run the following command:

```
1 cd connectedhomeip
2 scripts/examples/gn_build_
```



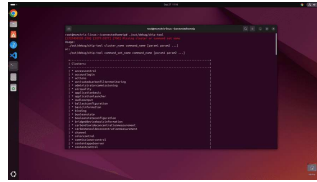
CHIP Tool environment set

Wait for the packages installation process to finish.

To check if the CHIP Tool runs correctly, execute the following command:

```
1 ./out/debug/chip-tool
```

As a result, the CHIP Tool starts and prints all available commands.



Environment test

The Use Case: Smart Outlet

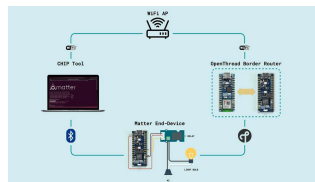
This section provides an example of commissioning and communication over Matter between an end-device node and the OTBR.

Setup

OTBR: Nano ESP32 + Nano Matter

CHIP Tool: running on a Linux PC (with Bluetooth capabilities)

End-device: Nano Matter or other boards compatible with the Silabs Arduino core

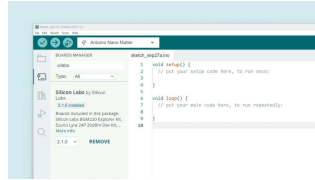


Solution architecture

End-Device Configuration

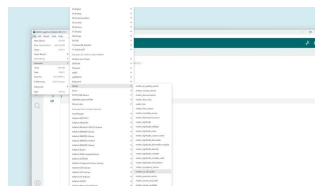
As the *end-device* we are going to use a Nano Matter configured as **Smart Outlet**.

Ensure the Silicon Labs boards package is installed in the



Silicon Labs board package

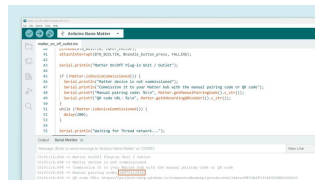
Go to **File > Examples > Matter > matter_on_off_outlet** and flash the example to the Nano Matter.



Smart outlet example

By default the example will use the built-in LED as an output, **i** you can easily update the code so it controls a GPIO to activate an external relay.

Once flashed, open the serial terminal and reset the board. Take note of the **manual pairing code**:



End-device Matter credentials

For example:

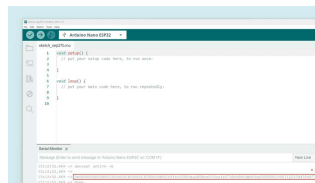
1 34970112332

Getting Thread Network Credentials

Connect through USB to the Nano ESP32 and run the following command from the Arduino IDE Serial Monitor:

```
1 dataset active -x
```


This command will display the active Thread network dataset as a hexadecimal string.



Thread network dataset
string

For example:

```
1 0e08000000000000100000000300
```

If an **error** appears, for example:

1 Error 23: NotFound

This means that there is no active Thread Network dataset configured. The following steps will help fix the error:

Check if the OpenThread is

```
1 state
```

If the response is **disabled**, it means OpenThread has not started. To start it, use:

```
1 ifconfig up
```

```
1 thread start
```

Then, try `dataset active -x` command again.

Now you should see the dataset string.

Matter Commissioning

Commissioning refers to setting up and integrating a new device into the Matter network.

In this case, commissioning will occur via **Bluetooth**, where the laptop or PC with the CHIP Tool installed will communicate with the end device using a Bluetooth connection.

Open the terminal on the system where the **CHIP Tool** is running, and execute the following command:

```
1 ./out/debug/chip-tool pair
```

device you are pairing (you can choose it freely according to your preference).

2 Replace

`thread-network-dataset` with the hexadecimal string representing the Thread network dataset.

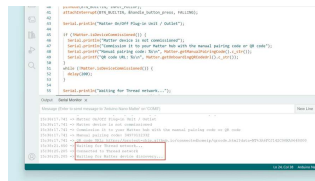
3 Replace

`end-device-pairing-code` with the manual pairing code for the end device.

Here is an example using the previously gathered parameters:

```
1 ./out/debug/chip-tool pair
```

If commissioning phase works fine, on the end-device serial monitor you will get the following:



Commissioning process

Now we are ready to control the Smart Outlet from the CHIP Tool system.

Final Result (Testing)

To control the Smart Outlet use the following command format:

```
1 ./out/debug/chip-tool <clu
2 # formatted command
3 ./out/debug/chip-tool onof
```

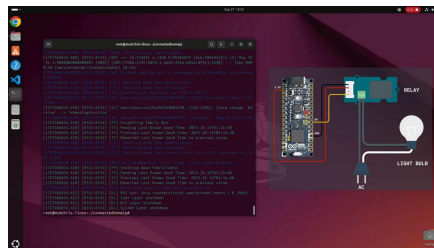
`onoff` : This specifies that the command pertains to the On/Off cluster, which controls the device's power state (on/off).

`toggle` : This command switches the current state of the device.

`1` : This is the Node ID of the device receiving the command.

`0x03` : This is the endpoint ID of the device (fixed to 0x03 for Silicon Labs device).

Every time you run the command, the Smart Outlet toggles, turning the connected load on or off.



Conclusion

This tutorial showed how to create an OpenThread Border Router using the Arduino Nano Matter and the Nano ESP32 alongside a Linux computer. With this solution you can easily integrate and control Matter Accessory Devices for Smart Homes or Industries.

Next Steps

Now that you know how to create your own Thread Border Router, you can continue developing a custom platform or application that uses the CHIP Tool API to control Matter end devices from a mobile app or web platform with user-friendly dashboards and interfaces.

**Suggest
changes**

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page [here](#).

**Need
support?**

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?